5to Laboratorio Tecnologías de microprocesamiento

Lujan Negrin, Sury Gonzalez, Gonzalo Gorosito
Universidad Tecnológica del Uruguay, ITRSO
Fray Bentos, Uruguay
{maria.negrin.a, sury.gonzalez, gonzalo.gorosito}@estudiantes.utec.edu.uy

I. Introducción

Los sistemas embebidos alimentados por baterías deben cumplir sus funciones dentro de restricciones temporales y, al mismo tiempo, minimizar la energía consumida. En un microcontrolador como el ATmega328P esto exige que el firmware no permanezca bloqueado esperando eventos, que la CPU y los periféricos se detengan cuando no son necesarios y que exista un mecanismo capaz de recuperar el sistema ante una falla de software, independientemente de la comunicación con el usuario.

En este laboratorio se integran estos aspectos en un único sistema basado en el ATmega328P programado en C. El firmware ejecuta tareas periódicas mediante planificación cooperativa sobre una base de tiempos de 1 ms, atiende eventos externos por interrupciones y organiza su comportamiento con una máquina de estados finitos de tres estados principales (ACTIVO, STANDBY y APAGADO), más un estado SEGURO para la atención de emergencias, implementada sin estructuras `switch-case`. Cada estado aplica un nivel distinto de ahorro energético mediante los modos Sleep del microcontrolador, la deshabilitación de periféricos con el registro PRR y el Watchdog Timer, utilizado tanto como despertador periódico como mecanismo de recuperación ante bloqueos. El sistema se supervisa y controla desde una aplicación móvil desarrollada en MIT App Inventor, que se comunica con un módulo HC-05 mediante un protocolo de tramas con verificación de integridad.

Finalmente, se caracteriza experimentalmente el sistema: se miden el período de las tareas y la corriente consumida en cada estado, se estima la autonomía del sistema alimentado por batería y se describe el procedimiento previsto para la caracterización energética avanzada.

II. Resumen

Este informe presenta el diseño, la implementación y la caracterización de un sistema embebido de bajo consumo basado en el microcontrolador ATmega328P, programado en lenguaje C con avr-gcc. El firmware se organiza en librerías modulares que proveen una base de tiempos con planificación cooperativa no bloqueante, adquisición analógica periódica, lectura no bloqueante de un sensor DHT11, manejo de una pantalla LCD por I2C, comunicación serie por interrupciones con búferes circulares y un protocolo de tramas con checksum para el enlace Bluetooth con un módulo HC-05. Sobre estas librerías se implementa una máquina de estados finitos con los estados ACTIVO, STANDBY, APAGADO y SEGURO, sin estructuras `switch-case`, cuyas transiciones dependen de la inactividad del usuario, de los pulsadores, de los comandos recibidos y de las solicitudes de emergencia. La gestión energética combina los modos Idle y Power-Down, la deshabilitación selectiva de periféricos mediante el registro PRR y el Watchdog Timer, configurado como fuente de despertar periódico y como mecanismo de reinicio ante bloqueos. Se desarrolla además una aplicación en MIT App Inventor para supervisar y controlar el sistema. Se verifica el funcionamiento en simulación y en la placa real, se mide el período de las tareas y se caracteriza el consumo de corriente de cada estado mediante una resistencia shunt y un sistema de adquisición propio, a partir del cual se estima la autonomía con una batería. Por último, se describe el procedimiento previsto para la caracterización energética avanzada del microcontrolador.

***Palabras clave*—ATmega328P, planificación cooperativa, máquina de estados finitos, modos Sleep, Watchdog Timer, PRR, Bluetooth, consumo energético.**

III. Objetivos

Objetivo general

Diseñar un sistema embebido de bajo consumo basado en el ATmega328P, con comunicación Bluetooth y supervisión mediante Watchdog Timer, y caracterizar experimentalmente su comportamiento temporal y energético.

Objetivos específicos

1. Implementar una arquitectura de firmware no bloqueante que integre planificación cooperativa, interrupciones, una máquina de estados finitos sin `switch-case` y comunicación Bluetooth mediante tramas.
2. Aplicar los modos Sleep, el registro PRR y el Watchdog Timer del ATmega328P en la gestión

energética y en la recuperación del sistema ante bloqueos.

3. Cuantificar el consumo de corriente y la precisión temporal del sistema en sus distintas configuraciones de funcionamiento.

IV. Marco Teórico

IV-A. Planificación cooperativa no bloqueante

En un esquema de planificación cooperativa, el programa principal recorre continuamente un lazo (*superloop*) en el que cada tarea se ejecuta solo cuando se cumple su período, medido a partir de una base de tiempos común generada por un temporizador. A diferencia de un sistema operativo de tiempo real, no existe expropiación: cada tarea se ejecuta hasta terminar y devuelve el control voluntariamente. Por ello, ninguna tarea puede utilizar esperas activas como `_delay_ms()`, ya que retrasaría a todas las demás; las operaciones largas se dividen en etapas cortas que avanzan en sucesivas pasadas del lazo.

Las interrupciones complementan este esquema: las rutinas de servicio (ISR) realizan solo el trabajo mínimo e imprescindible (incrementar un contador, almacenar un dato, activar una bandera) y el procesamiento se realiza en el programa principal. Las variables compartidas entre una ISR y el programa principal se declaran `volatile`, y las de más de un byte se leen en secciones atómicas, porque la CPU de 8 bits las accede en varias instrucciones que una interrupción podría separar.

IV-B. Máquinas de estados finitos

Una máquina de estados finitos (FSM) se define por un conjunto de estados, entradas, salidas, una función de estado siguiente y una función de salida. En el modelo de Moore las salidas dependen únicamente del estado, mientras que en el de Mealy dependen del estado y de las entradas [1]. En un microcontrolador, los estados representan situaciones estables del sistema, los eventos corresponden a pulsadores, vencimientos de temporizadores o datos recibidos, y las transiciones son las reglas que llevan de un estado a otro [1].

Además de la estructura `switch-case`, una FSM puede implementarse mediante condiciones explícitas o mediante una tabla de transiciones, en la que cada fila asocia un estado de origen y un evento con un estado de destino y, opcionalmente, una acción. La tabla separa la lógica de transición del código que la recorre, lo que facilita verificar que todas las combinaciones de estado y evento estén contempladas.

IV-C. Búferes circulares y protocolos de tramas

Un búfer circular es un arreglo de tamaño fijo con dos índices: la cabeza, donde el productor escribe, y la cola, donde el consumidor lee. Al llegar al final del arreglo, los índices vuelven al inicio. En la comunicación serie, la ISR de recepción actúa como productor y el programa principal como consumidor (y a la inversa en la transmisión), de modo que ninguno espera al otro y los datos no se pierden mientras el búfer tenga espacio. Si el tamaño es una potencia de dos, el retorno al inicio se resuelve con una máscara binaria.

Un protocolo de tramas organiza los bytes en mensajes delimitados por caracteres de inicio y fin, con campos de comando y dato y un campo de verificación (*checksum*) calculado a partir del contenido. El receptor descarta las tramas cuyo checksum no coincide, las que contienen caracteres inválidos y las que quedan incompletas al vencer un tiempo máximo entre caracteres, e informa el error al emisor.

Tabla I: Modos Sleep del ATmega328P estudiados [2]

Modo	**Relojes activos**	**Fuentes de despertar**
Idle	clk_{IO}, clk_{ADC}, clk_{ASY} (solo se detiene la CPU)	Cualquier interrupción: timers, USART, ADC, INT0/1, PCINT, TWI, WDT
ADC Noise Reduction	clk_{ADC}, clk_{ASY}	ADC, INT0/1, PCINT, TWI (dirección), Timer2 asíncrono, WDT
Power-Down	Ninguno (oscilador detenido)	INT0/1 por nivel, PCINT, TWI (dirección), WDT
Power-Save	clk_{ASY}	Igual que Power-Down, más Timer2 asíncrono

IV-D. Modos de bajo consumo del ATmega328P

El ATmega328P dispone de seis modos Sleep que se seleccionan con los bits SM2..0 del registro SMCR y se activan con el bit SE y la instrucción `SLEEP`. Cada modo detiene distintos dominios de reloj, por lo que difieren en el consumo, en los periféricos que siguen funcionando y en las fuentes capaces de despertar al microcontrolador [2]. La Tabla I resume los modos estudiados.

En Power-Down se detienen todos los relojes generados y solo los periféricos asíncronos permanecen activos [3]. Por ese motivo, las interrupciones INT0 e INT1 solo pueden despertar al microcontrolador configuradas por nivel bajo, ya que la detección de flancos requiere el reloj de E/S, y la USART no puede recibir datos. El modo Power-Save requiere un cristal de 32,768 kHz en los pines TOSC1/TOSC2 para operar el Timer2 de forma asíncrona. El modo ADC Noise Reduction reduce el ruido digital durante la conversión y la inicia automáticamente al entrar en él si el ADC está habilitado [2].

Durante Power-Down y Power-Save puede deshabilitarse además el detector de baja tensión (BOD) mediante una secuencia temporizada sobre los bits BODS y BODSE del registro MCUCR, lo que reduce el consumo cuando el BOD está habilitado por fusibles [2]. La biblioteca avr-libc provee macros para seleccionar el modo y realizar estas secuencias [4].

IV-E. Power Reduction Register (PRR)

El registro PRR permite detener el reloj de periféricos individuales para reducir el consumo, tanto en modo activo como en Idle. Cada bit corresponde a un periférico: PRADC (ADC), PRUSART0, PRSPI, PRTIM1, PRTIM0, PRTIM2 y PRTWI; el bit 4 está reservado. Al escribir un uno en un bit, el estado del periférico se congela y sus registros dejan de poder leerse o escribirse; al borrarlo, el periférico retoma el estado que tenía [2]. Como los recursos que estaba usando permanecen ocupados, el periférico debe deshabilitarse antes de detener su reloj; en particular, el ADC debe apagarse con el bit ADEN antes de activar PRADC. El comparador analógico no está incluido en PRR y se desactiva con el bit ACD del registro ACSR.

IV-F. Watchdog Timer

El Watchdog Timer (WDT) es un temporizador alimentado por un oscilador interno independiente de 128 kHz que, si no se reinicia periódicamente con la instrucción `WDR`, vence y provoca una acción. Su uso clásico es reiniciar el microcontrolador cuando el programa queda bloqueado, por ejemplo en un lazo infinito, con tiempos de hasta 8 s [3]. El intervalo se selecciona con los bits WDP3..0 del registro WDTCSR, entre aproximadamente 16 ms y 8 s [2], [3].

El WDT tiene tres modos de funcionamiento, definidos por los bits WDIE y WDE de WDTCSR [2]:

- **Interrupción** (WDIE = 1, WDE = 0): al vencer genera una interrupción y nunca reinicia el microcontrolador. Al ser asíncrono, puede despertarlo desde Power-Down.
- **Reset** (WDIE = 0, WDE = 1): al vencer reinicia el microcontrolador.
- **Interrupción y reset** (WDIE = 1, WDE = 1): el primer vencimiento genera una interrupción y el hardware borra WDIE, de modo que el siguiente vencimiento produce el reinicio. Permite guardar información antes del reset.

Para evitar cambios accidentales, la configuración exige una secuencia temporizada: se escriben en uno los bits WDCE y WDE y, dentro de los cuatro ciclos siguientes, se escribe la configuración definitiva. El fusible WDTON fuerza el modo reset de forma permanente. La causa de cada reinicio queda registrada en el registro MCUSR mediante los bits PORF (encendido), EXTRF (pin de reset), BORF (baja tensión) y WDRF (Watchdog). Tras un reinicio por WDT, el temporizador permanece habilitado con el intervalo mínimo, por lo que el firmware debe deshabilitarlo al comienzo de la ejecución [2], [4].

IV-G. Prescaler del reloj del sistema

El registro CLKPR divide la frecuencia del reloj del sistema por factores de 1 a 256, mediante una secuencia temporizada que habilita el cambio con el bit CLKPCE y escribe los bits CLKPS3..0 dentro de los cuatro ciclos siguientes [2]. Reducir la frecuencia disminuye el consumo dinámico de la CPU, pero altera la frecuencia de todos los periféricos síncronos: los períodos de los temporizadores, la velocidad de la USART y los retardos calculados a partir de `F_CPU` quedan escalados por el mismo factor y deben recalcularse.

IV-H. Medición y estimación del consumo energético

La corriente puede medirse a partir de la caída de tensión sobre una resistencia de bajo valor conectada en serie con la alimentación (resistencia shunt), aplicando la ley de Ohm:

$$I = \frac{V_{shunt}}{R_{shunt}} \tag{1}$$

Para un conjunto de N muestras de corriente tomadas a intervalos iguales, la corriente promedio es:

$$\bar{I} = \frac{1}{N}\sum_{k=1}^{N} I_k \tag{2}$$

y, si los intervalos no son iguales, debe ponderarse cada muestra por su duración. La carga consumida en un intervalo de duración T y la potencia media resultan:

$$Q = \int_0^T i(t)\,dt \approx \bar{I}\,T, \qquad \bar{P} = V_{CC}\,\bar{I} \tag{3}$$

Como primera aproximación, la autonomía de una batería de capacidad C (en mAh) que entrega una corriente promedio $\bar{I}$ (en mA) es:

$$t_{bat} = \frac{C}{\bar{I}} \quad \text{[h]} \tag{4}$$

Una estimación realista debe considerar además el perfil de descarga, la tensión mínima de funcionamiento, las pérdidas del regulador y el consumo de los demás componentes de la placa, que en un Arduino Uno pueden superar ampliamente al del microcontrolador. La reducción porcentual del consumo de una configuración respecto de una referencia se calcula como:

$$\text{Reducción}\,(\%) = \frac{\bar{I}_{ref} - \bar{I}_{opt}}{\bar{I}_{ref}} \times 100 \tag{5}$$

IV-I. MIT App Inventor

MIT App Inventor es un entorno de desarrollo visual para aplicaciones Android en el que la interfaz se diseña arrastrando componentes y el comportamiento se programa con bloques encastrables [5]. La comunicación Bluetooth clásica se realiza con el componente no visible *BluetoothClient*, que permite listar los dispositivos vinculados, conectarse a uno de ellos, enviar texto o bytes y consultar la cantidad de bytes disponibles para leerlos. Como no genera eventos al recibir datos, la recepción se resuelve consultando periódicamente con

un componente *Clock*, que también permite detectar la pérdida de comunicación cuando no llegan respuestas dentro de un tiempo determinado. El proyecto se exporta como un archivo `.aia`, que contiene el diseño y los bloques de la aplicación.

V. Materiales

- Placa Arduino Uno (microcontrolador ATmega328P a 16 MHz).
- Pantalla LCD 16×2 con módulo de comunicación I2C (PCF8574).
- Sensor de temperatura y humedad DHT11.
- Módulo Bluetooth HC-05.
- Potenciómetro.
- Dos pulsadores.
- Buzzer activo.
- LED y resistencias limitadoras.
- Resistencias para el divisor de tensión de la línea RXD del HC-05.
- Resistencia shunt de 0,1 Ω.
- Osciloscopio digital.
- Multímetro digital.
- Fuente de alimentación de laboratorio.
- Protoboard y cables de conexión.
- Teléfono inteligente con sistema operativo Android.
- Computadora con Microchip Studio (avr-gcc), Proteus y Git.
- MIT App Inventor.

VI. Procedimiento

VI-A. *Parte A*

VI-A1. Hardware: Se utilizó un Arduino Uno (ATmega328P a 16 MHz) con un potenciómetro en A2 (ADC), un sensor DHT11 en D4, una LCD 16×2 con adaptador I2C (PCF8574) en A4/A5, un buzzer activo en D9 y el LED integrado en D13. Hay dos pulsadores con resistencia de pull-up interna: D2 (INT0) y D3 (INT1, emergencia). La Tabla II resume la asignación de pines y la Fig. 1 muestra el esquema de conexiones.

Tabla II: Asignación de pines de la Parte A

Pin	**Función**
D2 (INT0)	Pulsador: despertar, buzzer y rearme
D3 (INT1)	Pulsador de emergencia
D4	DHT11
D9	Buzzer
D13	LED (parpadeo cada 200 ms en ACTIVO)
A2	ADC (potenciómetro)
A4 / A5	LCD por I2C

VI-A2. Máquina de estados: El sistema tiene cuatro estados: ACTIVO, STANDBY, APAGADO y SEGURO (Fig. 2). El estado seguro se agregó para atender la solicitud de emergencia. En esta parte solo se implementa

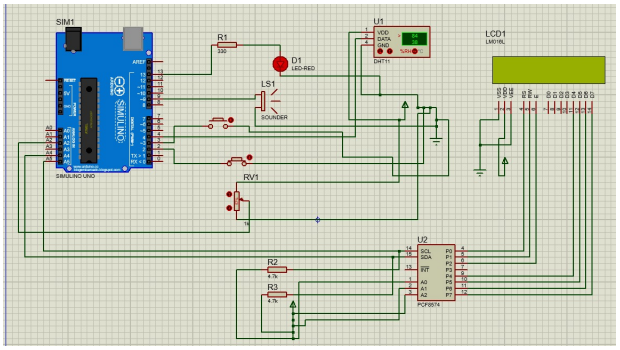


Figura 1: Esquema de conexiones de la Parte A

la lógica de la FSM: los modos Sleep y la gestión del registro PRR se aplican en la Parte B.

- Inicio: ACTIVO al encender.
- ACTIVO → STANDBY: 10 s sin actividad.
- STANDBY → APAGADO: 30 s contados desde la última actividad, no desde la entrada a STANDBY.
- STANDBY o APAGADO → ACTIVO: pulsador D2.
- Cualquier estado → SEGURO: pulsador D3.
- SEGURO → ACTIVO: pulsador D2 (rearme).

Se considera actividad cada pulsación válida de D2. La lectura periódica de los sensores no reinicia el contador de inactividad. En cada estado:

- ACTIVO: ejecuta las cinco tareas del sistema (LED, ADC, DHT11, LCD y buzzer) y muestra temperatura y valor del ADC en la LCD.
- STANDBY: apaga el LED, suspende las tareas periódicas y muestra "Esperando…".
- APAGADO: deshabilita el ADC, muestra "APAGADO" y apaga la retroiluminación de la LCD.
- SEGURO: apaga el LED y el buzzer y muestra "ESTADO SEGURO".

Como pide la consigna, no se usó switch-case. La FSM se implementó con una tabla de transiciones (Tabla III): cada fila tiene el estado de origen, una función de condición y el estado de destino. La función fsm_ejecutar() recorre la tabla y aplica la primera fila cuyo origen coincide con el estado actual (o es el comodín CUALQUIERA) y cuya condición se cumple. Al cambiar de estado se llama a la función de entrada correspondiente, guardada en un arreglo de punteros a función indexado por el estado. La emergencia es la primera fila de la tabla, por lo que tiene prioridad sobre las demás.

VI-A3. Planificación de tareas: Un planificador cooperativo ejecuta tareas periódicas a partir de una base de tiempos de 1 ms generada por el Timer0 (Tabla IV). Cada tarea es una función corta, sin esperas, descrita en una tabla con su período, el instante de su última ejecución, un puntero a la función y un indicador de activa; la FSM suspende o reanuda tareas cambiando

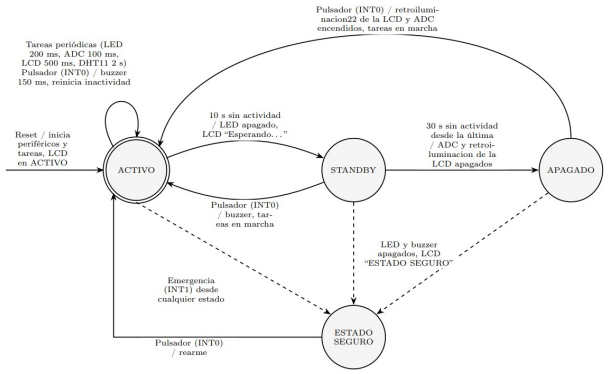


Figura 2: Máquina de estados de la Parte A

Tabla III: Tabla de transiciones de la Parte A (se aplica la primera fila que se cumple)

N.º	Estado origen	Condición (evento)	Estado destino	Acción al entrar al destino
1	Cualquiera	Emergencia (pulsador en INT1), si no está ya en estado seguro	SEGURO	Apaga LED y buzzer, suspende las tareas, enciende la retroiluminación de la LCD y muestra "ESTADO SEGURO/Emergencia"
2	ACTIVO	10 s sin actividad del usuario	STANDBY	Apaga el LED, suspende las tareas y muestra "Esperando..." en la LCD
3	STANDBY	Pulsación válida (INT0)	ACTIVO	Enciende la retroiluminación de la LCD, habilita el ADC, reinicia el contador de inactividad y reactiva las tareas
4	STANDBY	30 s sin actividad, contados desde la última actividad	APAGADO	Apaga el LED, suspende las tareas, deshabilita el ADC y apaga la retroiluminación de la LCD
5	APAGADO	Pulsación válida (INT0)	ACTIVO	Enciende la retroiluminación de la LCD, habilita el ADC, reinicia el contador de inactividad y reactiva las tareas
6	SEGURO	Pulsación válida (INT0), rearme	ACTIVO	Enciende la retroiluminación de la LCD, habilita el ADC, reinicia el contador de inactividad y reactiva las tareas

ese indicador. Las ISR solo marcan banderas o guardan datos (INT0 e INT1 con antirrebote de 50 ms y fin de conversión del ADC); el procesamiento se hace en el lazo principal.

Tabla IV: Tareas periódicas de la Parte A

Tarea	Período	Mecanismo
Base de tiempos	1 ms	Timer0 (ISR)
Parpadeo del LED	200 ms	Planificador
Lectura del ADC	100 ms	Timer1 y ADC (ISR)
Actualización LCD	500 ms	Planificador
Lectura del DHT11	2 s	Planificador
Pulsador	Inmediata	INT0
Buzzer	150 ms	Lazo principal

VI-A4. Registros configurados: La Tabla V resume la configuración. El Timer0 en modo CTC con prescaler 64 y OCR0A = 249 interrumpe cada $64 \cdot 250/16\,\mathrm{MHz} = 1$ ms. El Timer1 en modo CTC con prescaler 256 y OCR1A = 6249 interrumpe cada $256 \cdot 6250/16\,\mathrm{MHz} = 100$ ms y dispara la conversión del ADC. El ADC usa AVcc como referencia, el canal 2 y un prescaler de 128 (125 kHz), con interrupción al terminar. El Timer2 solo tiene reloj durante la ráfaga de lectura del DHT11, con prescaler 8 (0,5 μs por cuenta). INT0 e INT1 se disparan por flanco descendente.

Tabla V: Registros configurados

Periférico	Registro	Valor
Timer0	TCCR0A, TCCR0B, OCR0A	0x02, 0x03, 249
Timer1	TCCR1B, OCR1A	0x0C, 6249
ADC	ADMUX, ADCSRA	0x42, 0x8F
Timer2 (DHT11)	TCCR2B	CS21 (N = 8)
INT0 e INT1	EICRA, EIMSK	0x0A, 0x03

VI-A5. Cómo se evita el bloqueo entre tareas: No se usa _delay_ms() en el lazo principal. Las esperas se resuelven comparando contra millis(): el pulso de inicio de 20 ms del DHT11, el apagado del buzzer y los contadores de inactividad. La LCD escribe en un búfer en RAM y lcd_nb_tarea() envía como máximo cuatro caracteres por vuelta, de modo que ninguna llamada bloquea más de unos 1,5 ms. El único tramo largo es la ráfaga de lectura de los 40 bits del DHT11, de unos 4 ms y acotada por un tiempo máximo por nivel, con las interrupciones habilitadas. La inicialización de la LCD (aprox. 1 s) ocurre una sola vez al arrancar.

VI-A6. Verificación temporal del LED: Se midió el período del LED con un osciloscopio digital UNI-T UTD2052CEX, con la punta en D13 y la masa en GND. Se configuró 2 V/div, 500 ms/div y disparo por flanco ascendente en 2,56 V. Con los cursores de tiempo se midió ΔT entre dos flancos ascendentes separados por $N = 16$ ciclos. El período es $T = \Delta T/N$ y el error se calcula sobre el tiempo en alto, que debe ser 200 ms:

$$\text{Error}\,(\,\%) = \frac{T/2 - 200\,\text{ms}}{200\,\text{ms}} \times 100 \qquad (6)$$

Como el LED cambia de estado cada 200 ms, el período completo es de 400 ms.

VI-A7. Procedimiento de pruebas: Se verificó en el simulador (Proteus, con Simulino) y en la placa real. Se comprobó la actualización de la LCD, el parpadeo del LED, el cambio del ADC al mover el potenciómetro, el sonido del buzzer con D2, las transiciones por inactividad (10 s y 30 s) y las transiciones por D2 y D3.

VI-B. Parte B

VI-B1. Hardware: El sistema usa un Arduino Uno (ATmega328P a 16 MHz) con un potenciómetro en

A2 (ADC), un sensor DHT11 en D4, una LCD 16×2 con adaptador I2C en A4/A5, un buzzer en D9, el LED integrado en D13 y un módulo Bluetooth HC-05 conectado a la USART (D0 y D1). El RXD del HC-05 recibe a través de un divisor resistivo de 1 kΩ y 2 kΩ, porque trabaja a 3,3 V. Hay dos pulsadores con resistencia de pull-up interna: D2 (INT0) y D3 (INT1, emergencia). Un cable de D5 a GND se usa solo para simular un bloqueo en la prueba del WDT. La Tabla VI resume la asignación de pines y la Fig. 3 muestra el esquema utilizado en la simulación.

Tabla VI: Asignación de pines de la Parte B

Pin	Función
D0 / D1	USART hacia el HC-05 (9600 bps, 8N1)
D2 (INT0)	Pulsador local: despertar, buzzer, rearme
D3 (INT1)	Pulsador de emergencia
D4	DHT11
D5	Cable a GND para simular bloqueo
D9	Buzzer
D13	LED (parpadeo cada 200 ms en ACTIVO)
A2	ADC (potenciómetro)
A4 / A5	LCD por I2C

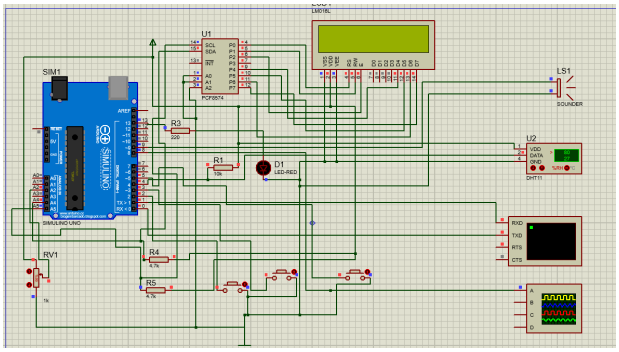


Figura 3: Esquema de conexiones de la Parte B en Proteus. El terminal virtual reemplaza al módulo HC-05.

VI-B2. Máquina de estados: El sistema tiene cuatro estados: ACTIVO, STANDBY, APAGADO y SEGURO (Fig. 4).

- Inicio: ACTIVO al encender; SEGURO si el reinicio fue provocado por el WDT.
- ACTIVO → STANDBY: 10 s sin actividad.
- STANDBY → APAGADO: 30 s desde el último evento válido.
- STANDBY o APAGADO → ACTIVO: pulsador D2; en STANDBY también una orden válida de la aplicación.
- Cualquier estado → SEGURO: pulsador D3, emergencia de la aplicación o reinicio por WDT.
- SEGURO → ACTIVO: D2 mantenido 1 s, o rearme desde la aplicación una vez apretado D2.

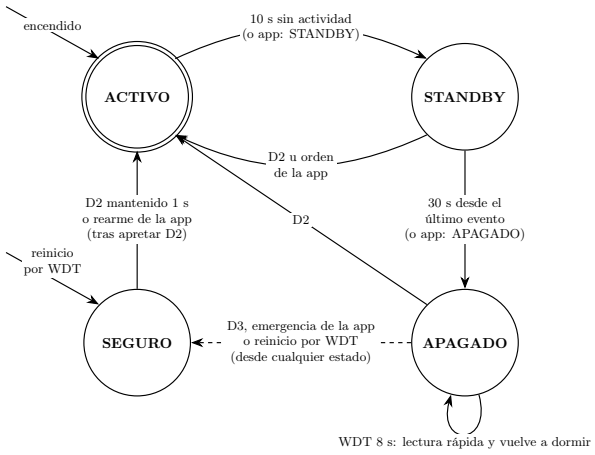


Figura 4: Máquina de estados de la Parte B

Como pide la consigna, no se usó `switch-case`. Se implementó con tres tablas de punteros a función indexadas por el estado (`tabla_entrar`, `tabla_ejecutar` y `tabla_dormir`). La función `cambiar_estado()` guarda el estado nuevo y llama a la función de entrada correspondiente. Los comandos del Bluetooth se despachan de la misma forma, con la tabla `tabla_cmd`.

VI-B3. Planificación de tareas: Un planificador cooperativo ejecuta cinco tareas periódicas: pulsador (20 ms), lectura del DHT11 (2 s), control de sensores (500 ms), actualización de la LCD (500 ms) y parpadeo del LED (200 ms). La base de tiempo es el Timer0 (1 ms). Las ISR de los pulsadores y de la USART solo marcan banderas o guardan datos; el procesamiento se hace en el lazo principal. Los pulsadores usan un antirrebote de 300 ms.

VI-B4. Gestión de energía:

- **ACTIVO:** no duerme y todos los periféricos están habilitados.
- **STANDBY:** modo Idle. Se apagan con PRR el ADC, el Timer0 y el Timer2. La base de tiempo pasa al Timer1 (100 ms).
- **APAGADO:** modo Power-Down. Se apagan con PRR el ADC, la USART, el TWI y los tres timers; la LCD se apaga y se retira la retroiluminación. El tiempo dormido se compensa al despertar.
- **SEGURO:** no duerme y se mantienen la comunicación y la LCD para informar.

El SPI se deshabilita desde el inicio, porque no se usa. La Tabla VII resume el estado de los periféricos.

Los periféricos se apagan en orden: primero se deshabilitan y después se les quita el reloj con PRR; se restauran en el orden inverso. No se usó una escritura indiscriminada de 0xFF en PRR, sino máscaras explícitas. Como la USART no puede recibir en Power-Down, la recuperación se hace con D2, D3 o el WDT.

Tabla VII: Estado de los periféricos según el modo de funcionamiento

Periférico	ACTIVO	STANDBY	APAGADO	SEGURO
ADC	ON	OFF	OFF	ON
USART0	ON	ON	OFF	ON
TWI (LCD)	ON	ON	OFF	ON
Timer0	ON	OFF	OFF	ON
Timer1	ON	ON	OFF	ON
Timer2	ON	OFF	OFF	ON
SPI	OFF	OFF	OFF	OFF

VI-B5. Watchdog:

- **Modo reset, 1 s** (ACTIVO, STANDBY y SEGURO): el lazo principal lo alimenta en cada vuelta; si el programa se bloquea, el WDT reinicia el micro.
- **Modo interrupción, 8 s** (APAGADO): despierta de Power-Down, hace una lectura rápida del ADC, informa el evento y vuelve a dormir, sin reiniciar.
- **Prueba A:** se deja el sistema en APAGADO y se verifica que el WDT lo despierte periódicamente sin reiniciarlo.
- **Prueba B:** en ACTIVO se une D5 a GND, lo que provoca un bucle infinito sin alimentar el WDT.

Para decidir si un reinicio fue provocado por el WDT, el programa debería leer la causa del reinicio registrada en MCUSR. En la placa utilizada, la lectura de MCUSR, la del registro r2 y una marca guardada en RAM no permitieron detectarlo. Por eso, antes de provocar el bloqueo se guarda una marca en la EEPROM, que al arrancar se lee y se borra para decidir si hay que pasar a SEGURO.

VI-B6. Protocolo de tramas: Se usa una trama ASCII de 10 caracteres: $<$, comando (2 dígitos hexadecimales), dato (4), checksum (2) y $>$. El checksum es la suma del comando y los dos bytes del dato, módulo 256. Una trama incompleta se descarta por tiempo de espera entre caracteres (100 ms en la placa; 10 s en la simulación, para poder escribirla a mano). El procesamiento se hace fuera de las ISR, con buffers circulares de recepción y de transmisión.

Las respuestas llevan el comando con el bit 7 en 1 (ACK = comando + 0x80). Los datos de sensores se envían como 82.. (ADC) y 83.. (temperatura). Los eventos usan 90CCVV.. con estos códigos: 1 temperatura alta, 2 emergencia de la aplicación, 3 emergencia local, 4 reinicio por WDT, 5 despertar por WDT, 6 cambio de estado y 7 arranque. Los errores usan EE000N.. (Tabla IX).

Los comandos se validan antes de ejecutarse. La aplicación no puede anular una emergencia sola: el rearme se rechaza hasta que se cumple la condición local (apretar D2).

VI-B7. Aplicación móvil: La aplicación se desarrolló en MIT App Inventor (Fig. 5). Se conecta a los dispositivos Bluetooth ya emparejados, consulta el estado cada pocos segundos, muestra el estado y las lecturas, envía comandos, muestra los eventos y avisa si pasan más de 7 s sin respuesta. Tiene un botón de emergencia, que es complementario y no sustituye al pulsador físico. La Fig. 6 muestra el diagrama de comunicación entre la aplicación, el HC-05 y el micro. El estado que muestra la aplicación se actualiza tanto con la consulta periódica como con los eventos de cambio de estado que envía el micro.

Tabla VIII: Comandos del sistema

Código	Comando	Ejemplo
0x01	Consultar estado	01000001
0x02	Pedir sensores	02000002
0x03	Cambiar estado (0 ACTIVO, 1 STANDBY, 2 APAGADO)	03000104
0x04	Actuadores (LED parpadeo, buzzer)	04010106
0x05	Emergencia	05000005
0x06	Consultar alarmas	06000006
0x07	Rearme	07000007

Tabla IX: Códigos de error

N	Error
1	Checksum incorrecto
2	Formato inválido
3	Trama incompleta
4	Comando desconocido
5	Comando rechazado



Figura 5: Interfaz de la aplicación desarrollada en MIT App Inventor.

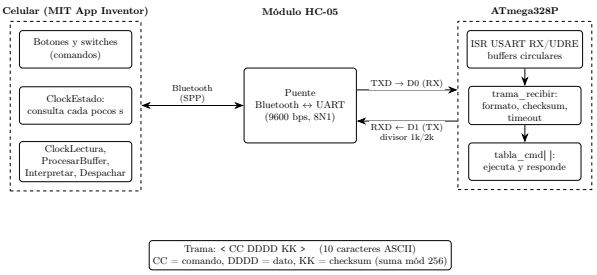


Figura 6: Diagrama de comunicación entre la aplicación, el HC-05 y el micro.

VI-B8. Procedimiento de pruebas: Se verificó en el simulador (Proteus) y en la placa real.

- **Simulación:** se cargó el programa compilado en el ATmega328P de Proteus, con la dirección I2C de la LCD correspondiente al modelo del simulador y el tiempo de espera de tramas en 10 s. Las tramas se escribieron a mano en el terminal virtual. Se probaron los comandos y los errores de comunicación.
- **Placa real, estados:** con el sistema en reposo se observaron las transiciones por inactividad (10 s y 30 s) y los despertares con D2.
- **Placa real, aplicación:** se emparejó el HC-05 con el teléfono, se conectó la aplicación y se probaron la consulta de estado, las lecturas, los cambios de estado, los actuadores, la emergencia y el rearme.
- **Prueba A:** con el sistema en APAGADO se observaron los despertares periódicos del WDT, la lectura rápida del ADC y el regreso a Power-Down.
- **Prueba B:** con el sistema en ACTIVO se unió D5 a GND y se observó el reinicio y el estado posterior.

VI-C. Parte C

VI-C1. Configuraciones medidas: Se midió la corriente que consume el sistema de la Parte B en cuatro configuraciones (Tabla X). El firmware es el mismo de la Parte B, con dos macros de compilación: `C_APAGADO_CON_WDT` selecciona si en APAGADO el Watchdog despierta al micro cada 8 s (1) o queda deshabilitado (0), y `C_USAR_PRR` selecciona si se apagan periféricos con el registro PRR (1) o no (0). Las transiciones son las de la Parte B: ACTIVO pasa a STANDBY a los 10 s sin actividad y STANDBY pasa a APAGADO a los 30 s desde el último evento.

Tabla X: Configuraciones de la Parte C

Config.	Estado	Modo del micro	WDT	PRR
A	ACTIVO	sin sleep	reset 1 s	1
B	STANDBY	Idle	reset 1 s	1
C	APAGADO	Power-Down	int. 8 s	1
D	APAGADO	Power-Down	no	1

Macros: `C_APAGADO_CON_WDT` (C = 1, D = 0) y `C_USAR_PRR`. Referencia (Actividad 6): A, B y C con PRR = 0.

VI-C2. Montaje de medición: El sistema medido es el de la Parte B completo: Arduino Uno a 16 MHz con LCD I2C, módulo HC-05, DHT11, pulsadores, buzzer y LED en D13. Se alimentó con 5,0 V desde una fuente de laboratorio. En el retorno de la alimentación, entre el GND del sistema medido y el GND común del instrumento, se colocó una resistencia shunt de 4,7 Ω. Toda la corriente del sistema pasa por ella y produce una tensión $V_{shunt} = I \cdot R_{shunt}$. Esa tensión se amplifica con un LM358 no inversor de ganancia $G = 1+R_f/R_g = 1+1\,\mathrm{k\Omega}/1\,\mathrm{k\Omega} = 2$. Entre la salida del LM358 y la entrada A0 de un segundo Arduino hay un divisor de 1 kΩ en serie y 10 kΩ a GND, que atenúa la señal a 10/11. El 5 V del segundo Arduino no se conectó al circuito medido; solo se comparten los GND.

Se usó una shunt de 4,7 Ω y no de 0,1 Ω porque con 90 mA esta última produce solo 9 mV, que tras la ganancia de 2 y el divisor equivalen a unas 3 cuentas del ADC, una resolución insuficiente. Con 4,7 Ω caen unos 0,4 V a 90 mA, que se miden con claridad. El costo es que el circuito medido recibe unos 0,4 V menos de alimentación.

El segundo Arduino funciona solo como registrador: lee A0 cada 5 ms (200 muestras por segundo) con una referencia de ADC igual a su alimentación, y envía por serie a 115 200 baudios una línea `micros,adc` con el valor crudo. La referencia del ADC, $V_{REF} = 5{,}11$ V, se midió con el multímetro entre los pines 5V y GND del Arduino registrador. La corriente se calcula en la computadora a partir de cada cuenta del ADC:

$$I = \frac{\left(\dfrac{ADC \cdot V_{REF}}{1023} \cdot 1{,}1 - V_{off}\right)/2}{R_{shunt}} \qquad (7)$$

donde 1,1 deshace el divisor, 2 deshace la ganancia y V_{off} es el offset del amplificador. Una cuenta del ADC equivale a

$$\Delta I = \frac{V_{REF}}{1023} \cdot \frac{1{,}1}{2\,R_{shunt}} = \frac{5{,}11}{1023} \cdot \frac{1{,}1}{2 \cdot 4{,}7} \approx 0{,}6\ \mathrm{mA} \quad (8)$$

El ADC es de 10 bits (1024 niveles) y guarda el resultado en los registros ADCH:ADCL; una cuenta vale unos 5,0 mV en A0.

La temperatura ambiente durante las mediciones fue de 24 a 25 °C. La alimentación del sistema medido es la de la fuente de laboratorio, 5,0 V nominales; esa tensión no se midió con el multímetro en el pin 5V del Arduino medido. La shunt de 4,7 Ω es de 5 anillos (amarillo, violeta, negro, plata y marrón), lo que corresponde a $470 \times 0{,}01 = 4{,}7\ \Omega$ con tolerancia de $\pm 1\,\%$. Las resistencias de 1 kΩ y de 10 kΩ también son de $\pm 1\,\%$ (último anillo marrón).

VI-C3. Calibración del offset: El LM358 entrega una pequeña tensión aunque la entrada sea cero. Antes de cada corrida se puentearon las patas de la shunt y el script promedió 3 s de lecturas para obtener V_{off}, que luego resta de todas las muestras. Después se retiró el puente y se comenzó a medir.

VI-C4. Software de adquisición: Un script de Python recibe las muestras en un hilo aparte, convierte cada una a corriente con la ecuación (7), la grafica en vivo junto con un promedio móvil de 0,25 s y guarda tres archivos al cerrar: el CSV con todas las muestras, un resumen por estado (mínimo, máximo, media, desviación, carga, potencia y energía) y la imagen de la gráfica. El estado de cada muestra lo marca el operador con el teclado (1 ACTIVO, 2 STANDBY, 3 APAGADO, 4 EMERGENCIA). La carga se integra con la regla del trapecio entre muestras consecutivas y se descartan los intervalos de más de 1 s.

VI-C5. Procedimiento de medición:

1. Con el sistema medido encendido, calibrar el offset como se indicó.
2. **Configuraciones A, B y C (una sola corrida):** resetear el Arduino medido, apretar 2 a los 10 s (STANDBY) y 3 a los 30 s (APAGADO), y a los 45 s pasar el sistema a EMERGENCIA (estado seguro). Cerrar con Q. Las tres configuraciones salen de la misma captura.
3. **Configuración D:** con `C_APAGADO_CON_WDT = 0`, resetear el Arduino y esperar más de 30 s para que ya esté en APAGADO. Calibrar, apretar 3 al comenzar la medición y registrar unos 25 s. Para D se usa solo el tramo APAGADO.
4. **Referencia sin PRR (Actividad 6):** con `C_USAR_PRR = 0` y `C_APAGADO_CON_WDT = 1`, repetir la secuencia de A, B y C. D no se repitió sin PRR porque en Power-Down los relojes de los periféricos ya están detenidos.
5. Para cargar cada firmware se desconectó D0 del HC-05, porque comparte la USART con el USB, y se reconectó al retirar el cable.

En todas las corridas se mantuvo el mismo montaje: mismo HC-05, misma shunt, mismo LED en D13 y sin USB conectado al sistema medido.

VI-C6. Cálculos: Con N muestras equiespaciadas (5 ms) de un estado de duración T:

$$\bar{I} = \frac{1}{N}\sum_{k=1}^{N} I_k, \qquad Q \approx \bar{I}\,\frac{T}{3600}\,[\mathrm{mAh}],$$
$$\bar{P} = V_{CC}\,\bar{I}, \qquad E = V_{CC}\,Q \qquad (9)$$

con $V_{CC} = 5{,}0$ V. La reducción porcentual respecto de una referencia es la de la ecuación (5). La autonomía ideal es $t = C/\bar{I}$ (ecuación (4)) y la realista usa el

VI-D. Parte D

Por limitaciones de tiempo, el Problema D no llegó a ejecutarse experimentalmente. A continuación se describe el procedimiento previsto para cada actividad, de

modo que la caracterización pueda reproducirse con el firmware y las librerías desarrolladas en los problemas anteriores.

VI-D1. Montaje de medición: Para todas las actividades se utilizaría el montaje del Problema C, con la resistencia shunt en serie con la alimentación de 5 V. Debido a que en Power-Down la corriente del microcontrolador es del orden de los microamperios, la caída sobre un shunt de 0,1 Ω sería inferior a la resolución del osciloscopio; por ello, para los modos de menor consumo se propone emplear el multímetro en su escala de μA, conectado en serie, o un shunt de mayor valor verificando que la caída de tensión no afecte la alimentación. Además, se registraría por separado el consumo de la placa Arduino Uno completa y, en la medida de lo posible, el del ATmega328P alimentado directamente, ya que el regulador, el conversor USB-serie y el LED de encendido de la placa consumen corriente en todos los modos y enmascaran el ahorro del microcontrolador.

VI-D2. Comparación de modos Sleep: Se programaría una prueba que, sin tareas activas, entre en forma repetida en un único modo Sleep, seleccionado con `energia_dormir(DORMIR_IDLE)`, `energia_dormir(DORMIR_ADC)` o `energia_power_down()`, y se mediría la corriente promedio en cada caso bajo las mismas condiciones de hardware. Para cada modo se documentarían las partes del microcontrolador que permanecen activas, los periféricos detenidos, las fuentes de despertar y las configuraciones adicionales necesarias, según la Tabla I. El modo Power-Save no se estudiaría, porque requiere un cristal de 32,768 kHz en los pines TOSC1/TOSC2, que en el Arduino Uno están ocupados por el cristal de 16 MHz [2].

VI-D3. Impacto de los periféricos mediante PRR: Se partiría de una configuración de referencia con todos los periféricos habilitados y la CPU en modo activo, y se deshabilitaría un único periférico por experimento mediante `energia_prr_apagar()` con su máscara explícita (`PRR_ADC`, `PRR_USART0`, `PRR_SPI`, `PRR_TIM0`, `PRR_TIM1`, `PRR_TIM2` y `PRR_TWI`), deshabilitando antes el periférico correspondiente. Luego se evaluarían combinaciones de periféricos, repitiendo cada configuración en modo activo y en Idle. En cada prueba se registrarían los periféricos habilitados, el modo Sleep, la corriente promedio y las observaciones, y se verificaría que el periférico deshabilitado no fuera necesario para la temporización de la prueba. La reducción respecto de la referencia se calcularía con (5).

VI-D4. Tiempo de despertar: El canal 1 del osciloscopio se conectaría al pulsador de INT0 y el canal 2 al pin de prueba PB0, configurado para cambiar de estado como primera instrucción de la rutina de interrupción. Con el disparo del osciloscopio en el flanco del pulsador, se mediría el intervalo entre ambas señales para las transiciones Idle $\rightarrow$ Activo, ADC Noise Reduction $\rightarrow$ Activo y Power-Down $\rightarrow$ Activo, repitiendo cada medición al menos diez veces para obtener los tiempos mínimo, promedio y máximo. Se espera que Idle y ADC Noise Reduction despierten en pocos ciclos de reloj, mientras que Power-Down debe esperar además el arranque del oscilador de cristal, cuya duración depende de los fusibles CKSEL y SUT [2]. En Power-Down, INT0 se configura por nivel bajo, por lo que el resultado también incluiría la latencia de la detección asíncrona.

VI-D5. Caracterización del Watchdog Timer: Se configuraría el WDT en modo interrupción con `wdt_modo_interrupcion()` para los intervalos nominales de 16 ms, 250 ms, 1 s y 8 s, conmutando PB0 dentro de la rutina de interrupción, que es lo suficientemente breve como para no afectar la medición. El período se mediría con el osciloscopio en al menos diez ciclos por configuración y se compararía con el valor nominal mediante:

$$\text{Error}\,(\,\%) = \frac{|T_{medido} - T_{nominal}|}{T_{nominal}} \times 100 \qquad (10)$$

Para el modo reset se configuraría `wdt_modo_reset()`, se marcaría con PB0 el inicio del bloqueo simulado y se mediría el tiempo hasta el reinicio, identificado por el primer cambio del pin al arrancar el firmware. Se dejaría explícito en cada medición si el WDT opera como fuente de interrupción o como mecanismo de reset. Como el oscilador de 128 kHz del WDT depende de la tensión y la temperatura [2], se esperan desviaciones respecto de los valores nominales.

VI-D6. Consumo frente a frecuencia de reloj: Mediante `energia_reloj_div()` se aplicarían los prescalers 1, 2, 8, 64 y 128, que corresponden a 16 MHz, 8 MHz, 2 MHz, 250 kHz y 125 kHz, y se mediría la corriente en estado activo ejecutando un lazo sin periféricos. Como `F_CPU` no se modifica, se verificaría que el período del LED se multiplica por el factor de división y que la USART deja de funcionar a 9600 bps. Para mantener la funcionalidad habría que recalcular los registros: a 2 MHz, por ejemplo, la base de 1 ms requiere un prescaler de 8 con OCR0A = 249 y la USART requiere UBRR0 = 12, mientras que a 250 kHz y 125 kHz no es posible obtener 9600 bps con un error aceptable, y la comunicación con el DHT11 tampoco sería viable porque sus pulsos son de decenas de microsegundos.

VI-D7. Análisis comparativo y comparación con el datasheet: Con los datos obtenidos se elaborarían, en Python, gráficas de consumo frente al modo Sleep, a la configuración de periféricos y a la frecuencia de reloj, de tiempo de despertar frente al modo Sleep y de consumo con y sin optimizaciones, utilizando escala logarítmica

cuando las corrientes difieran en varios órdenes de magnitud. Finalmente, los resultados se contrastarían con las curvas del fabricante [2], considerando la tensión de alimentación, la frecuencia, la temperatura, la configuración de periféricos, el consumo de los componentes de la placa y la resolución e incertidumbre de los instrumentos, distinguiendo en todo momento los valores teóricos, los medidos y los estimados.

VII. Resultados

VII-A. Parte A

VII-A1. Funcionamiento de la FSM (simulación): En Proteus la LCD mostró el estado ACTIVO con la temperatura y el valor del ADC. Tras 10 s sin actividad mostró "Esperando…" y al apretar D3 mostró "ESTADO SEGURO". El modelo de LCD de Proteus no tiene retroiluminación, por lo que en APAGADO solo se aprecia el mensaje "APAGADO" y no el apagado de la luz.

VII-A2. Medición del período del LED: La Tabla XI muestra la medición con el osciloscopio. Entre dos flancos ascendentes separados por 16 ciclos se midió $\Delta T = 6{,}410$ s, lo que da un período de 400,6 ms, un tiempo en alto de 200,3 ms y una frecuencia de 2,496 Hz. El error respecto del objetivo de 200 ms es de $+0{,}16\,\%$. Los cursores del osciloscopio avanzan de a 10 ms, por lo que ΔT tiene una incertidumbre de ± 10 ms y el período de unos $\pm 0{,}6$ ms ($\pm 0{,}16\,\%$); el error medido es del mismo orden que la resolución del instrumento. En la captura (Fig. 7) no se aprecian variaciones del ancho de los pulsos a la resolución de la pantalla.

Tabla XI: Medición del período del LED (objetivo: 200 ms en alto, 400 ms de período)

Ciclos	ΔT (s)	T (ms)	$T/2$ (ms)	Error (%)
16	6,410	400,6	200,3	$+0{,}16$

VII-A3. Pulsador, buzzer y emergencia: Cada pulsación válida de D2 hace sonar el buzzer 150 ms en cualquier estado y reinicia el contador de inactividad. D3 lleva a SEGURO desde cualquier estado y apaga el LED y el buzzer. En SEGURO, D2 vuelve a ACTIVO.

VII-B. Parte B

VII-B1. Comunicación (simulación): Se verificó la respuesta ante errores enviando tramas a mano desde el terminal virtual de Proteus (Tabla XII y Fig. 8).

El error de formato se provocó con el carácter de fin de línea del terminal, que llega dentro de una trama ya iniciada. La respuesta a la trama incompleta apareció después del tiempo de espera de 10 s configurado para la simulación.

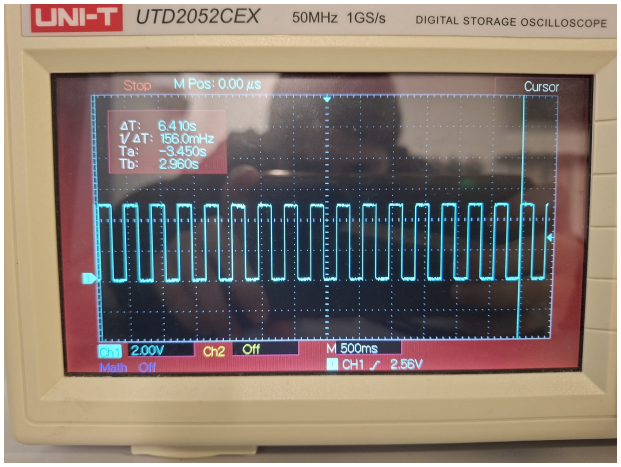


Figura 7: Captura del osciloscopio: 16 ciclos del LED (2 V/div, 500 ms/div)

Tabla XII: Respuesta ante tramas con errores (simulación)

Prueba	Enviado	Respuesta
Checksum incorrecto	01000002	EE0001EF (error 1)
Formato inválido	01000 + fin de línea	EE0002F0 (error 2)
Trama incompleta	010000 sin cerrar	EE0003F1 (error 3)
Comando desconocido	09000009	EE0004F2 (error 4)
Comando rechazado	03000306	EE0005F3 (error 5)

VII-B2. Comunicación con la aplicación (placa real): La aplicación se conectó al HC-05, mostró el estado y las lecturas (ADC y temperatura), envió los comandos y recibió los eventos (Fig. 9a). Al pedir el rearme en SEGURO antes de apretar D2, el micro respondió con el error 5, como se diseñó (Fig. 9b). En APAGADO la aplicación informó "Sin respuesta del micro", porque la USART está apagada (Fig. 9c).

VII-B3. Estados de energía: Con el sistema en reposo se observó el paso a STANDBY a los 10 s (la LCD muestra "Esperando...") y a APAGADO a los 30 s (la LCD muestra "APAGADO" y se apaga la retroiluminación). D2 despierta a ACTIVO (Fig. 10). No se midió el consumo.

VII-B4. Prueba B: WDT como reset: Al unir D5 a GND, el programa dejó de alimentar el WDT y la placa se reinició. Después del reinicio, la LCD mostró EMERGENCIA y la aplicación recibió la alarma de reinicio por WDT. Con D5 todavía a GND el sistema permanece en SEGURO; al retirar el cable y mantener D2 durante 1 s vuelve a ACTIVO.

Como se explicó en el procedimiento, la causa del reinicio no pudo leerse de MCUSR ni de r2 en esta placa, y una marca en RAM tampoco alcanzó. La prueba funcionó usando una marca guardada en la EEPROM. Esta marca solo se escribe antes del bloqueo provocado con D5, por lo que el método detecta ese bloqueo y no un

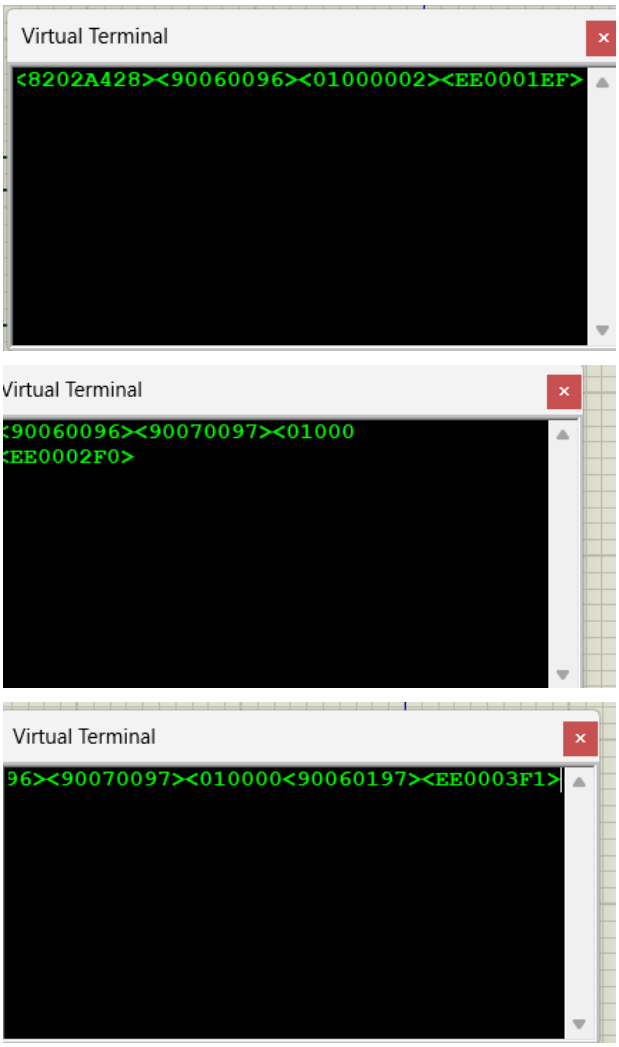


Figura 8: Terminal virtual de Proteus: respuesta ante (a) checksum incorrecto, (b) formato inválido y (c) trama incompleta.

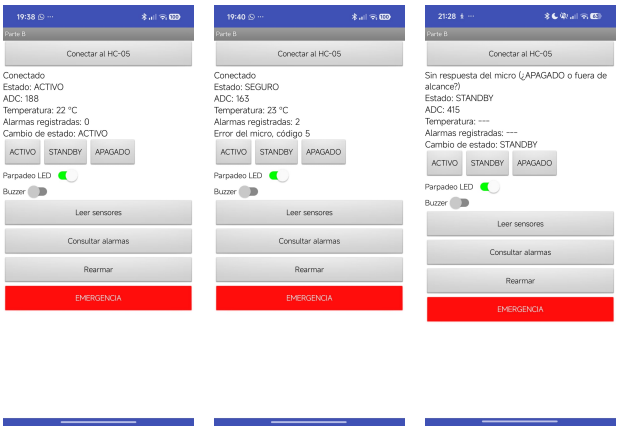
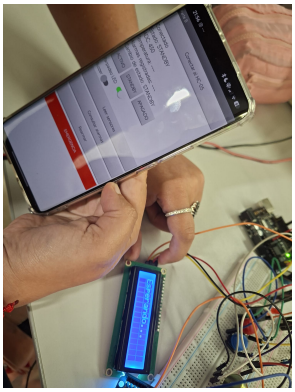


Figura 9: Aplicación: (a) conectada, con estado, lecturas y evento; (b) error 5 al pedir el rearme sin haber apretado D2; (c) aviso de comunicación perdida en APAGADO.



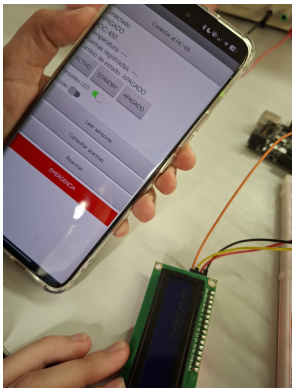


Figura 10: LCD en (a) STANDBY y (b) APAGADO.

reinicio por WDT originado en otro punto del programa.

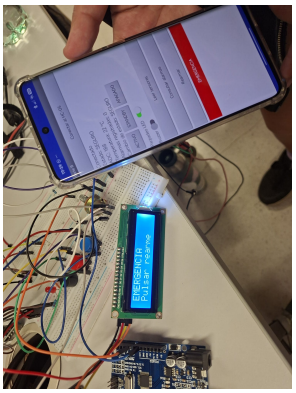


Figura 11: Prueba B en la placa real: LCD en EMERGENCIA/SEGURO tras el reinicio.

Limitación en el simulador. En Proteus el sistema envió la marca previa al bloqueo (9006FF95) y quedó congelado, sin reiniciarse (Fig. 12). No se verificó la causa; es consistente con que el simulador no reinicia el micro por vencimiento del WDT. La evidencia de esta prueba es la ejecución en la placa real.

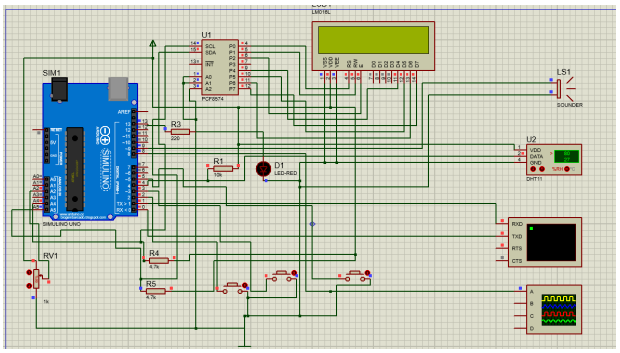


Figura 12: Prueba B en Proteus: el terminal recibe la marca previa al bloqueo y la LCD queda congelada.

VII-B5. Emergencia y rearme: D3 y el botón de emergencia/seguro de la aplicación llevan a SEGURO. El rearme de la aplicación es rechazado (error 5) hasta que se aprieta D2 una vez; mantener D2 durante 1 s vuelve a ACTIVO.

VII-C. Parte C

VII-C1. Montaje e incertidumbre: La resolución es de 0,6 mA por cuenta (ecuación (8)). La shunt, las resistencias de la ganancia (1 kΩ) y las del divisor (1 kΩ y 10 kΩ) son de $\pm 1\,\%$ (último anillo marrón). En el peor caso eso suma unos $\pm 2{,}2\,\%$ sobre la corriente (1 % de la shunt, 1 % de la ganancia y 0,2 % del divisor). A eso se agrega un error absoluto de unos $\pm 2{,}4$ mA: unas 2 cuentas del ADC (1,2 mA) y la variación del offset entre corridas (1,2 mA). Sumando ambos términos, la incertidumbre es de unos $\pm 5\,\%$ en ACTIVO (86,7 mA) y de unos $\pm 6\,\%$ en APAGADO (57,2 mA), mayor a corrientes bajas porque pesa el término absoluto. El offset calibrado fue de unos 0,01 a 0,02 V según la corrida. Entre corridas repetidas con el mismo firmware se observó una variación de unos 3 mA, que es la que limita las comparaciones entre corridas (PRR y WDT).

VII-C2. Medición por estados: La Tabla XIII muestra los resultados de la captura principal (PRR = 1) y de la configuración D (donde se mide únicamente el estado APAGADO). A, B y C salen de la misma captura; D salió de una corrida aparte solo en ese estado.

Tabla XIII: Corriente por configuración (PRR = 1; en D solo estado APAGADO)

Config.	Estado	T (s)	Mín (mA)	Máx (mA)	Media (mA)
A	ACTIVO	10,19	73,9	148,2	86,7
B	STANDBY	20,01	54,7	119,0	75,8
C	APAGADO con WDT	14,67	54,1	103,2	57,2
D	APAGADO sin WDT (solo APAGADO)	24,73	56,6	104,5	59,0
	EMERGENCIA	9,96	57,0	130,6	84,4

Los mínimos de ACTIVO, STANDBY y EMERGENCIA vienen de unas pocas muestras del borde entre estados (unos 0,25 s): el estado se asigna por el instante en que se aprieta la tecla y el escalón real de corriente cae un poco antes.

VII-C3. Registro temporal: Se guardó un CSV con 10 967 muestras en 54,83 s y las columnas tiempo (s), tensión de shunt (V), corriente (mA) y estado. La *tensión de shunt* es la calculada sobre la resistencia a partir del ADC y del amplificador, no una lectura directa. La Fig. 13 muestra la corriente en función del tiempo.

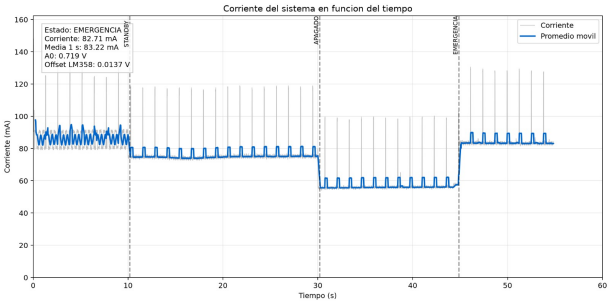


Figura 13: Corriente del sistema en función del tiempo para los cambios de estado. Las líneas verticales marcan los cambios de estado.

- **Funcionamiento activo (0 a 10 s):** base de unos 80 mA, con fluctuaciones entre unos 80 y 91 mA y media de 86,7 mA.
- **Transiciones a Sleep:** a los 10 s la corriente baja un escalón, a unos 75 mA (STANDBY, CPU en Idle). A los 30 s baja a unos 55 mA (APAGADO, CPU en Power-Down).
- **Picos:** pulsos de unos 45 a 55 mA sobre la base, cada 1,28 s y con la misma altura en todos los estados. Siguen presentes con la CPU dormida, por lo que no los produce el micro. Es consistente con el escaneo periódico del HC-05 mientras espera una conexión, aunque no se verificó retirando su alimentación.
- **Menor consumo:** APAGADO, de 30 a 45 s, con una base de unos 55 mA. No baja a microamperios porque la medición incluye la LCD, el HC-05, el regulador de la placa y el resto de los componentes.
- **Tareas periódicas:** en ACTIVO la corriente alterna con un período de unos 0,4 s, consistente con el LED que cambia de estado cada 200 ms. En STANDBY y APAGADO esa oscilación desaparece. De 45 s en adelante la base sube a unos 83 mA (EMERGENCIA).
- **Despertares del WDT:** duran mucho menos que el período de muestreo de 5 ms y no se distinguen en la captura.

VII-C4. Consumo promedio: La Tabla XIV aplica la ecuación (9) a cada estado. Por ejemplo, en ACTIVO: $\bar{I} = 86{,}71$ mA con $N = 2038$, $Q \approx 86{,}71 \cdot 10{,}19/3600 = 0{,}2453$ mAh y $\bar{P} = 5{,}0 \cdot 86{,}71 = 433{,}5$ mW. La carga del trapecio coincide con $\bar{I}\,T/3600$ hasta la cuarta cifra.

Tabla XIV: Consumo promedio por estado ($V_{CC} = 5{,}0$ V, reducción respecto de ACTIVO)

Estado	T (s)	$\bar{I}$ (mA)	Q (mAh)	$\bar{P}$ (mW)	E (mWh)	Reducción (%)
ACTIVO	10,19	86,71	0,2453	433,5	1,227	referencia
STANDBY	20,01	75,80	0,4212	379,0	2,106	12,6
APAGADO con WDT	14,67	57,20	0,2330	286,0	1,165	34,0
APAGADO sin WDT	24,73	59,05	0,4056	295,2	2,028	31,9

Q y E dependen de cuánto duró cada tramo; para comparar estados conviene usar la corriente y la potencia media. El mayor ahorro lo ofrece APAGADO, de 32 a 34 % respecto de ACTIVO, e Idle ahorra un 12,6 %. La potencia se calculó con la tensión nominal de la fuente, 5,0 V, por lo que incluye la disipada en la shunt; el sistema medido recibe unos 0,4 V menos

VII-C5. Autonomía: La Tabla XV estima la duración de una batería de 2400 mAh si el sistema permaneciera siempre en cada estado.

Tabla XV: Autonomía estimada con $C = 2400$ mAh

Estado	I (mA)	Ideal (h)	Realista (h)
ACTIVO	86,7	27,7	23,5
STANDBY	75,8	31,7	26,9
APAGADO con WDT	57,2	42,0	35,7
APAGADO sin WDT	59,0	40,6	34,5

La estimación realista usa el 85 % de la capacidad (2040 mAh) y tiene una incertidumbre en la corriente de ± 5 a $\pm 6\,\%$. Los valores incluyen el HC-05, la LCD, el regulador y el resto de la placa: esa corriente base es la diferencia entre el microcontrolador solo y el sistema completo. Un ciclo de uso real, que mezcla estados, queda entre los valores de ACTIVO y de APAGADO. Si el buzzer o el LED funcionan en el uso real, su corriente se suma aparte. Si la batería entrara por Vin habría que sumar la corriente propia del regulador y la pérdida por la caída de tensión. Cuatro pilas NiMH recién cargadas llegan a unos 5,6 a 5,8 V, por encima de los 5,5 V recomendados para el ATmega328P a 16 MHz; se resuelve con un diodo Schottky en serie, o con una celda Li-ion y un boost a 5 V. Los tiempos escalan en proporción a la capacidad elegida.

VII-C6. Efecto del PRR y del WDT: La Tabla XVI compara cada estado con y sin PRR (respecto de la captura con PRR de la Actividad 3), y la Tabla XVII muestra la reducción respecto de la referencia sin optimizaciones (ACTIVO sin PRR, 85,4 mA). Para la configuración sin WDT (D) se mide únicamente el estado APAGADO.

Tabla XVI: Corriente media con y sin PRR

Estado	Sin PRR (mA)	Con PRR (mA)	Diferencia
ACTIVO	85,4	86,7	$+1{,}6$ %
STANDBY	76,4	75,8	$-0{,}8$ %
APAGADO	55,2	57,2	$+3{,}7$ %
EMERGENCIA	86,5	84,4	$-2{,}5$ %

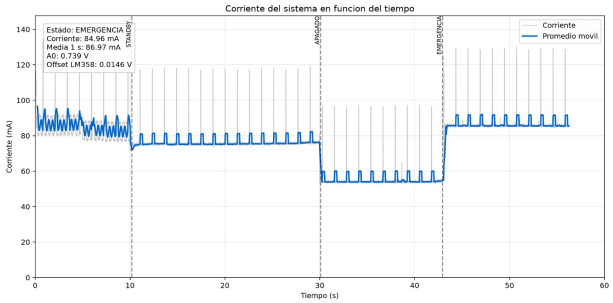


Figura 14: Corriente del sistema en función del tiempo sin utilizar el registro PRR.

Tabla XVII: Reducción respecto de ACTIVO sin PRR (85,4 mA)

Configuración	I (mA)	Reducción (%)
ACTIVO sin PRR	85,4	referencia
ACTIVO con PRR	86,7	$-1{,}6$
Idle sin PRR	76,4	10,5
Idle con PRR	75,8	11,2
Power-Down con WDT, sin PRR	55,2	35,4
Power-Down con WDT, con PRR	57,2	33,0
Power-Down sin WDT, con PRR (solo APAGADO)	59,0	30,8

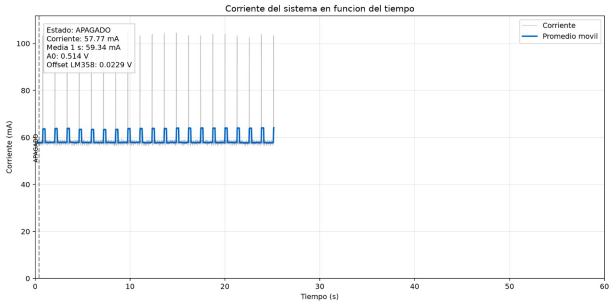


Figura 15: Corriente del sistema en función del tiempo para la configuración sin Watchdog (solo estado APAGADO).

El PRR no produce un cambio medible: las diferencias son de 0,6 a 2,1 mA (de 1 a 4 %) y cambian de signo según el estado. Para comparar dos corridas no cuenta la tolerancia de las resistencias, que afecta por igual a ambas, sino la variación entre corridas repetidas, de unos 3 mA (dos corridas de D con el mismo firmware dieron bases de 53,8 y 57,2 mA). Como todas las diferencias son menores que esa variación, no se pueden atribuir al PRR. Es esperable, porque el ahorro del PRR es del orden de décimas de mA según la hoja de datos y una cuenta del ADC equivale a 0,6 mA. Del mismo modo, la diferencia entre APAGADO con y sin WDT (57,2 y 59,0 mA, 1,8 mA) tampoco es atribuible al WDT: es menor que la variación entre corridas y sus despertares consumen mucho menos de lo que resuelve la medición. Los ahorros reales provienen de los modos Idle y Power-Down.

VII-C7. Limitaciones:

- Se mide el sistema completo y no solo el ATmega328P, por eso APAGADO queda en unos 57 mA.
- La shunt de 4,7 Ω resta unos 0,4 V a la alimentación a 90 mA y disipa unos 35 mW.
- El muestreo de 5 ms no resuelve los despertares del WDT.
- El estado se asigna por el instante en que se aprieta la tecla, lo que desplaza unos 0,25 s los bordes entre estados.
- La variación entre corridas (unos 3 mA) limita las comparaciones de PRR y de WDT.

VII-D. Parte D

No se obtuvieron resultados experimentales del Problema D, ya que no llegó a ejecutarse por limitaciones de tiempo. El procedimiento previsto para su realización se describe en la Sección VI-D.

VIII. Conclusiones

Se logró implementar una arquitectura de firmware no bloqueante en la que las tareas periódicas, la atención de los pulsadores, la comunicación Bluetooth y la máquina de estados funcionan de forma concurrente desde el punto de vista funcional. La combinación de una base de tiempos por interrupción, rutinas de servicio mínimas y tareas cortas que nunca esperan permitió que el parpadeo del LED mantuviera su período con un error del orden de la resolución del osciloscopio, aun mientras se actualizaba la LCD y se leía el DHT11. Organizar el firmware en librerías independientes facilitó probar cada parte por separado antes de integrarla, y la implementación de la FSM mediante tablas de transiciones y de punteros a función, en lugar de `switch-case`, resultó una alternativa clara y fácil de verificar. El protocolo de tramas detectó todos los errores de comunicación ensayados y permitió que el sistema validara los comandos antes de ejecutarlos, impidiendo que la aplicación anulara una emergencia sin la intervención local.

Los modos Sleep, el registro PRR y el Watchdog Timer se aplicaron de forma coordinada con los estados de la FSM. El WDT en modo reset recuperó el sistema ante un bloqueo y lo llevó a un estado seguro de forma independiente de la comunicación Bluetooth, mientras que en modo interrupción permitió despertar periódicamente al microcontrolador desde Power-Down sin reiniciarlo. Durante la implementación surgieron limitaciones que no son evidentes en la teoría: la USART no puede recibir en Power-Down, por lo que la recuperación desde ese estado depende de los pulsadores o del WDT; el bootloader de la placa impidió identificar la causa del reinicio a partir de MCUSR, lo que obligó a registrarla en la EEPROM; y el simulador no reproduce el reinicio por WDT, por lo que esa prueba solo pudo validarse en la placa real.

La caracterización del consumo mostró que, a nivel del sistema completo, los modos de bajo consumo del microcontrolador producen ahorros moderados: alrededor de un 13 % en Idle y de un 34 % en Power-Down respecto del estado activo. La corriente no desciende al orden de los microamperios que especifica el fabricante porque la medición incluye el HC-05, la LCD, el regulador y el resto de los componentes de la placa, que en este sistema consumen más que el propio ATmega328P. Por la misma razón, el efecto del PRR y del despertar periódico del WDT quedó por debajo de la resolución y de la repetibilidad de la medición. De esto se desprende que optimizar el firmware no alcanza para reducir el consumo de forma significativa: también es necesario desconectar o controlar la alimentación de los periféricos externos y prescindir de los componentes de la placa de desarrollo, aspectos que determinan la autonomía real con una batería.

Por limitaciones de tiempo no se realizó la caracterización avanzada del Problema D, aunque se dejó definido su procedimiento. Su ejecución requeriría instrumentos capaces de resolver corrientes del orden de los microamperios y, preferentemente, medir el microcontrolador fuera de la placa Arduino, lo que permitiría contrastar los resultados con las especificaciones del fabricante.

Apéndice A
Repositorio

El código fuente desarrollado en este laboratorio, junto con el historial de versiones de cada problema, se encuentra disponible en el siguiente repositorio público de GitHub:

https://github.com/gonzalogorosito-collab/Labs-MIcro-2026

Los videos que demuestran el funcionamiento de los sistemas desarrollados en este laboratorio se encuentran disponibles en la siguiente carpeta de Google Drive:

https://drive.google.com/drive/folders/18f-QBDo8Ov-le6IYr6dMr_XD4ZQrbC6B?usp=drive_link

Referencias

[1] J. Martínez and M. Díaz, "Máquinas de Estados," Material del curso Tecnologías de Microprocesamiento, Ingeniería en Mecatrónica, UTEC, ITR Suroeste, 2026.

[2] Microchip Technology Inc., *ATmega48A/PA/88A/PA/168A/PA/328/P megaAVR Data Sheet*, 2020, DS40002061B.

[3] J. Martínez and M. Díaz, "WDT – Sleep," Material del curso Tecnologías de Microprocesamiento, Ingeniería en Mecatrónica, UTEC, ITR Suroeste, 2026.

[4] AVR Libc Project, *AVR Libc User Manual*, https://avrdudes.github.io/avr-libc/avr-libc-user-manual/, módulos `<avr/sleep.h>`, `<avr/wdt.h>` y `<avr/power.h>`.

[5] Massachusetts Institute of Technology, "MIT App Inventor," https://appinventor.mit.edu.